

自然语言处理 (2025 春季学期)

六、预训练语言模型进阶



大纲

- 模型优化
- 长文本处理
- 模型蒸馏与压缩
- 生成模型





模型优化

随着以GPT、BERT为代表的预训练语言模型的提出，很多的工作集中在进一步优化预训练语言模型，使之在各类自然语言处理任务上获得更好的效果。

接下来，将围绕**模型优化**方面介绍五个有代表性的预训练语言模型：XLNet、RoBERTa、ALBERT、ELECTRA和MacBERT，主要介绍模型的设计思路及具体的建模方法。



XLNet: Transformer-XL Net

□ 语言模型通常被分为两大类:

— 自回归语言模型 (**Auto-Regressive Language Model, ARLM**)

自回归语言模型是一种最常用的语言模型。传统基于N-gram的语言模型就属于这个类别。该类模型需要基于给定的历史文本序列预测下一个单词的概率，其建模方法如下所示。

$$\log P(x) = \sum_{i=1}^N \log P(x_i | x_{1:i-1})$$

— 自编码语言模型 (**Auto-Encoding Language Model, AELM**)

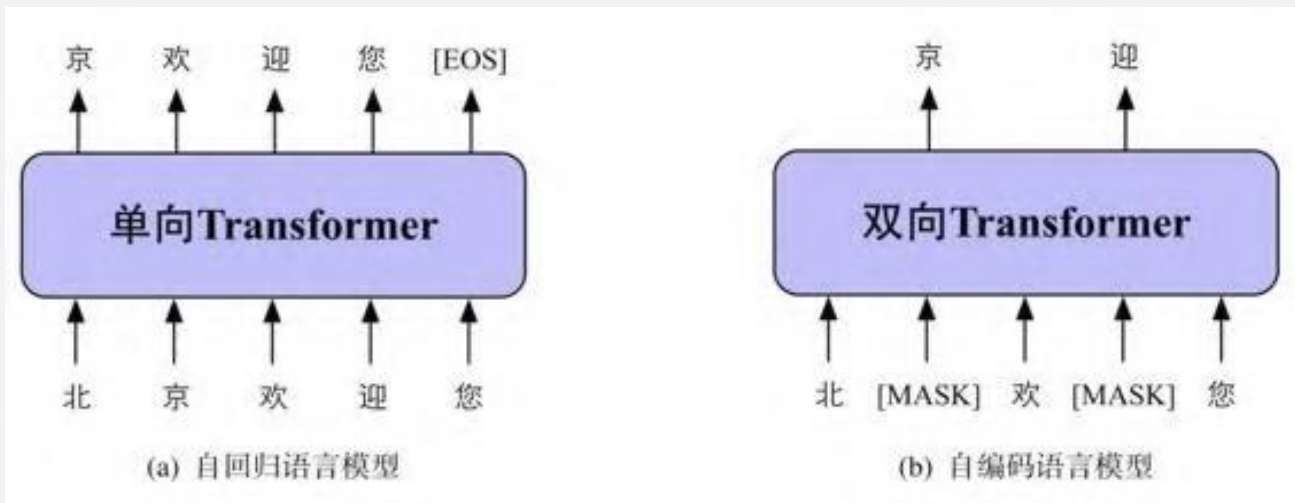
对于自编码语言模型，其目标是通过上下文重构被掩码的单词。第7章重点介绍的BERT就属于这个类别，其建模公式表示为：

$$\log P(x|\hat{x}) = \sum_{i=1}^N m_i \log P(x_i|\hat{x})$$



XLNet: Transformer-XL Net

- 语言模型通常被分为两大类：
 - 自回归语言模型（Auto-Regressive Language Model, ARLM）
 - 自编码语言模型（Auto-Encoding Language Model, AELM）





XLNet: Transformer-XL Net

□ XLNet的特点

- 使用了自回归语言模型结构，使得各个单词的预测存在依赖性，同时避免了自编码语言模型中引入人造标记[MASK]的问题；

解决了BERT中存在的“预训练-精调”不一致的问题

- 引入了自编码语言模型中的双向上下文，能够利用更加丰富的上下文信息，而不像传统的自回归语言模型只能利用单向的历史信息；

提出了一种可以捕获双向上下文的基于自回归的语言建模方法

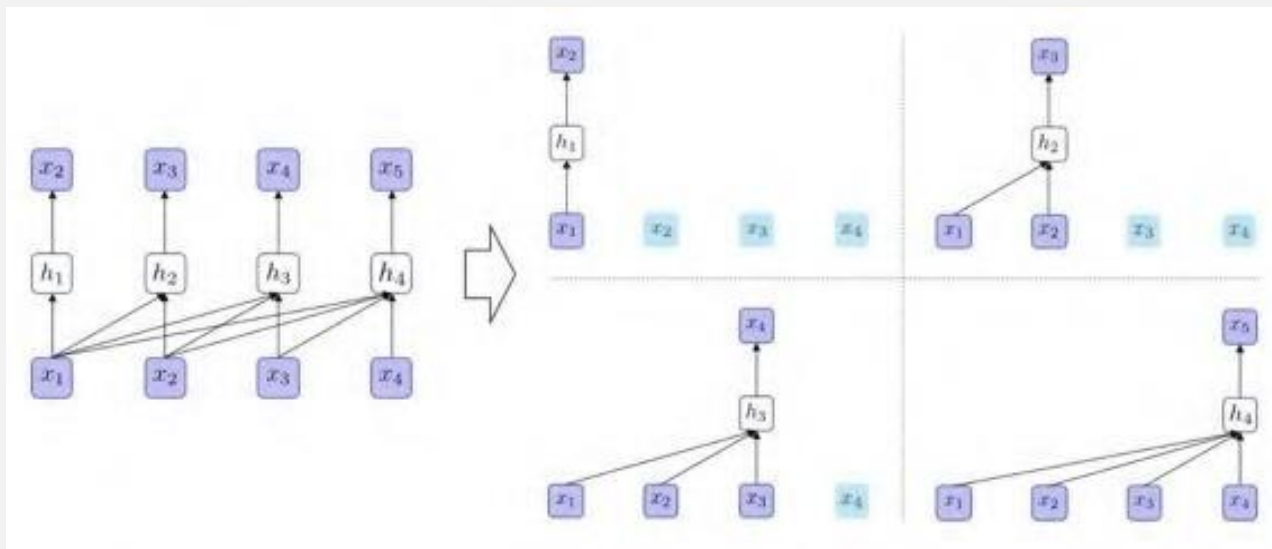
- 使用了Transformer-XL作为主体框架，相比传统的Transformer拥有更好的性能。



XLNet: Transformer-XL Net

□ 改进1：排列语言模型

- 先回顾传统的自回归语言模型，其中预测每一个单词需要依赖其历史词。



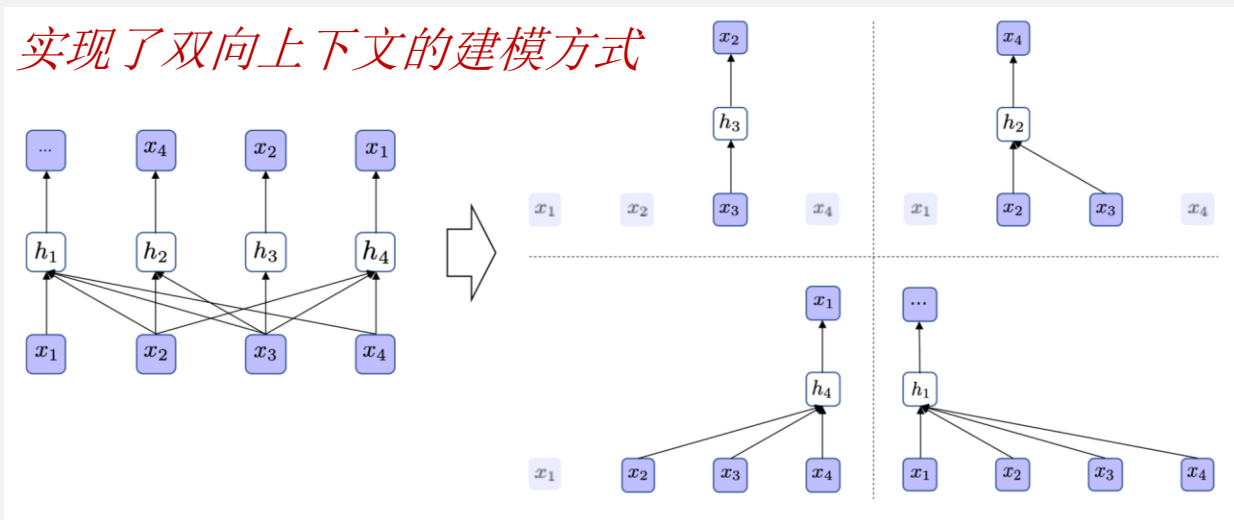


XLNet: Transformer-XL Net

□ 改进1：排列语言模型

– 改变建模顺序：3→2→4→1

– 句子建模 $P(x) = P(x_3)P(x_2|x_3)P(x_4|x_3x_2)P(x_1|x_3x_2x_4)$





XLNet: Transformer-XL Net

□ 改进1：排列语言模型

- 从所有可能的排列当中均匀采样一种排列顺序（factorization order）
- 最大化对数似然函数

$$\mathbb{E}_{z \sim \mathbb{Z}_N} [\log P(x|z)] = \mathbb{E}_{z \sim \mathbb{Z}_N} \left[\sum_{i=1}^N P(x_{z_i} | x_{1:i-1}, z_i) \right]$$

- 假设通过标准的Softmax函数建模下一个词的概率分布

$$P(x_{z_i} = x | x_{1:i-1}) = \frac{\exp(\mathbf{v}_x^\top \mathbf{h}_{x_{1:i-1}})}{\sum_{x'} \exp(\mathbf{v}_{x'}^\top \mathbf{h}_{x_{1:i-1}})}$$

- 对于不同的目标位置 z_i ，上式式总会产生一样的概率分布，这将无法满足建模要求。可做如下变动：

$$P(x_{z_i} = x | x_{1:i-1}) = \frac{\exp(\mathbf{v}_x^\top g(x_{1:i-1}, z_i))}{\sum_{x'} \exp(\mathbf{v}_{x'}^\top g(x_{1:i-1}, z_i))}$$





XLNet: Transformer-XL Net

□ 改进2：双流注意力机制

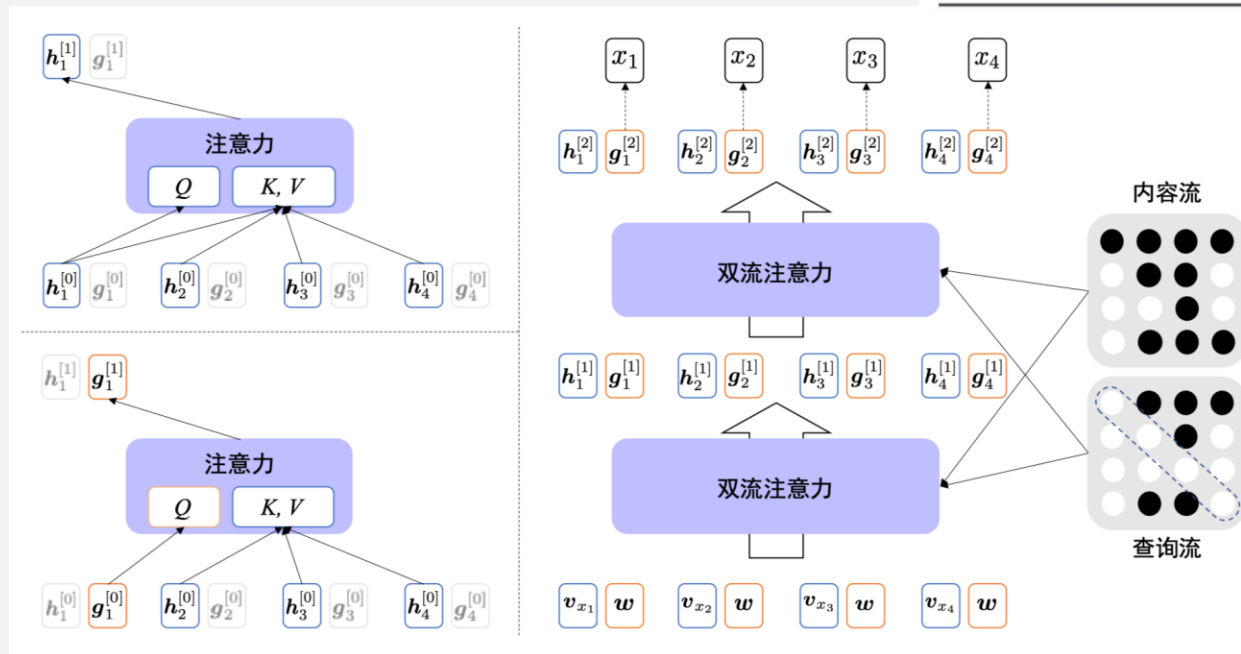
- 如何构造这样一种函数 g
- 若使用原始注意力机制构造 g ，存在问题：
 x_{z_i} 的编码是否需要编码 x_{z_i} 的内容
 - 若编码，则存在泄露问题
 - 若不编码，则无法提供信息作为上下文
- 提出了双流自注意力机制（Two-stream Self-attention），使用两套表示方法解决上述矛盾
 - 内容表示（Content Representation）：即原始的Transformer表示方法，可以同时建模单词及其上下文；
 - 查询表示（Query Representation）：能建模上下文信息以及目标位置但不能看到单词。



XLNet: Transformer-XL Net

改进2: 双流注意力机制

当前预测	当前历史	内容流置一元素	查询流置一元素
3 → 2 → 4 → 1	$\emptyset$	$M_{3,3}^h$	$\emptyset$
3 → 2 → 4 → 1	x_3	$M_{2,3}^h, M_{2,2}^h$	$M_{2,3}^g$
3 → 2 → 4 → 1	x_3, x_2	$M_{4,3}^h, M_{4,2}^h, M_{4,4}^h$	$M_{4,3}^g, M_{4,2}^g$
3 → 2 → 4 → 1	x_3, x_2, x_4	$M_{1,3}^h, M_{1,2}^h, M_{1,4}^h, M_{1,1}^h$	$M_{1,3}^g, M_{1,2}^g, M_{1,4}^g$





□ 概述

- 探究BERT的各个设计环节，包括掩码策略、NSP的有效性、训练步数等
- 提出CC-News数据集，证明使用更多数据可以进一步提升预训练模型效果

□ 静态掩码 vs. 动态掩码

- 目的：增加掩码语言模型中的随机性，提高文本的利用率
- 静态掩码：模型训练之前（数据预处理）阶段决定哪些词被mask，即BERT使用的方法
- 动态掩码：模型训练的过程中决定哪些词被mask，提高数据的复用效率

轮数
↓

went to the store → went to the [MASK]
went to the store → went to the [MASK]
went to the store → went to the [MASK]
went to the store → went to the [MASK]
went to the store → went to the [MASK]

静态掩码

went to the store → went to the [MASK]
went to the store → went to [MASK] store
went to the store → go to the store
went to the store → went to the store
went to the store → went [MASK] the store

动态掩码





RoBERTa

□ NSP预训练任务是否是必要的？

- 实验结果表明，舍弃NSP任务可以获得微弱性能提升

文本对输入+NSP

句子对输入+NSP

跨文档整句输入

文档内整句输入

Model	SQuAD 1.1/2.0	MNLI-m	SST-2	RACE
<i>Our reimplementaion (with NSP loss):</i>				
SEGMENT-PAIR	90.4/78.7	84.0	92.9	64.2
SENTENCE-PAIR	88.7/76.2	82.9	92.1	63.0
<i>Our reimplementaion (without NSP loss):</i>				
FULL-SENTENCES	90.4/79.1	84.7	92.5	64.8
DOC-SENTENCES	90.6/79.7	84.7	92.7	65.6
BERT _{BASE}	88.5/76.3	84.3	92.8	64.3
XLNet _{BASE} (K = 7)	-/81.3	85.8	92.7	66.1
XLNet _{BASE} (K = 6)	-/81.0	85.6	93.4	66.7





□ 其他优化

- 使用更大的批次大小以及更多数据
- 适当增加预训练步数，可以进一步提升预训练效果
- 目前广泛被认可的实证结论：训练预训练模型的关键之一是选用大的批次大小

数据名称	文本类型	大小
BookCorpus	故事	16 GB
Wikipedia	百科	
CC-News	新闻	76 GB
OpenWebText	社区问答	38 GB
Stories	故事	31 GB

批次大小	训练步数	PPL	MNLI-m	SST-2
256	1M	3.99	84.7	92.5
	125K	3.68	85.2	93.1
2,048 (2K)	250K	3.59	85.3	94.1
	500K	3.51	85.4	93.5
8,192 (8K)	31K	3.77	84.4	93.2
	63K	3.60	85.3	93.5
	125K	3.50	85.8	94.1



□ 最终RoBERTa采用了上面所有优点

– 预训练任务

- 动态掩码技术（Dynamic Masking）
- 使用整句输入，舍弃NSP损失（Full-Sentences without NSP loss）

– 预训练实验设置

- 使用更大的批次大小：256 → 8192
- 更大的byte-level BPE词表（SentencePiece）：30k → 50K

在原始BERT中，采用了一个30K大小的WordPiece词表，这是一种基于字符级别（Char-level）的BPE词表。

弊端是，如果输入文本无法通过词表中的WordPiece子词进行拼接组合，则会映射到“unknown”这种未登录词标识。

RoBERTa模型使用了SentencePiece分词器，并且将词表大小扩大至50K。采用SentencePiece这种字节级别（Byte-level）BPE词表的好处是能够编码任意输入文本，因此不会出现未登录词的情况。



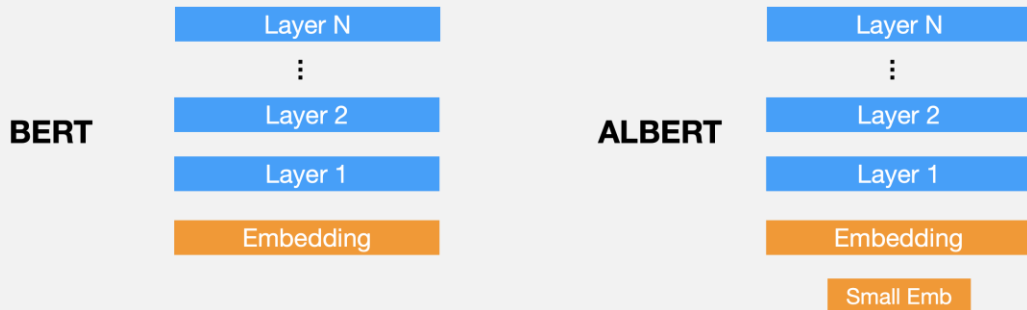
ALBERT

□ 概述

- 提出一种更加小巧（从参数量的角度）的预训练
- 两种主要技术：词向量因式分解、跨层参数共享

□ 词向量因式分解（Factorized Embedding Parameterization）

- 在BERT中， $\text{embedding_size} == \text{hidden_size}$
- 在ALBERT中， $\text{embedding_size} < \text{hidden_size}$





ALBERT

□ 词向量因式分解

- 利用词向量因式分解后

$$O(V \times H) \quad \longrightarrow \quad O(V \times E + E \times H)$$

- 举例： $V=30000, H=2014, E=128$

BERT

$$V * H = 30000 * 1024 = 30,720,000$$

>

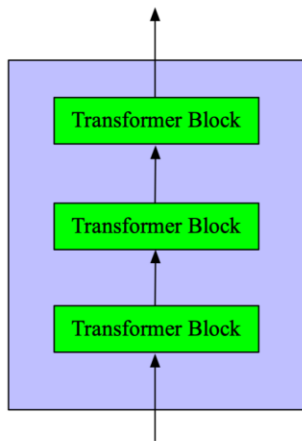
ALBERT

$$V * E + E * H = 30000 * 128 + 128 * 1024 = 3,971,072$$

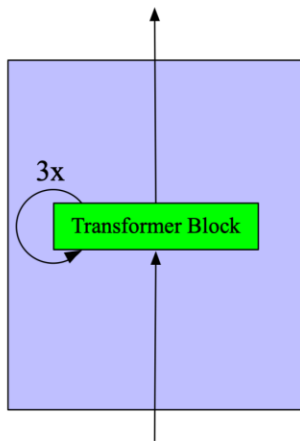


□ 跨层参数共享

- Transformer每层的参数是共享的，即只需要存一份参数，与层数无关
- 训练时：虽然参数共享，但每层的梯度是不同的，仍然需要额外空间存储
- 推断时：前向计算过程仍然要一层层展开，并不能节省推断时间



(a) 无跨层参数共享



(b) 有跨层参数共享

对比项目	参数量	磁盘占用	内存占用	前向传播时间
无跨层参数共享（3层）	3x	3x	3x	3x
有跨层参数共享（3层）	1x	1x	3x	3x

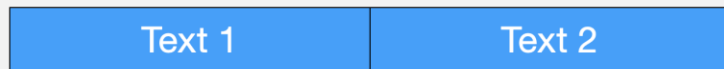




□ 句子顺序预测 (Sentence Order Prediction, SOP)

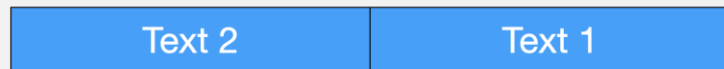
- NSP任务实际隐含了“连贯性”和“主题”预测
- 然而，判断两段文本的主题是否一致是比较容易的
- ALBERT提出句子顺序预测任务
 - 正样本：与BERT相同，由两个连续的文本段组成

+



- 负样本：交换两个连续文本段的顺序

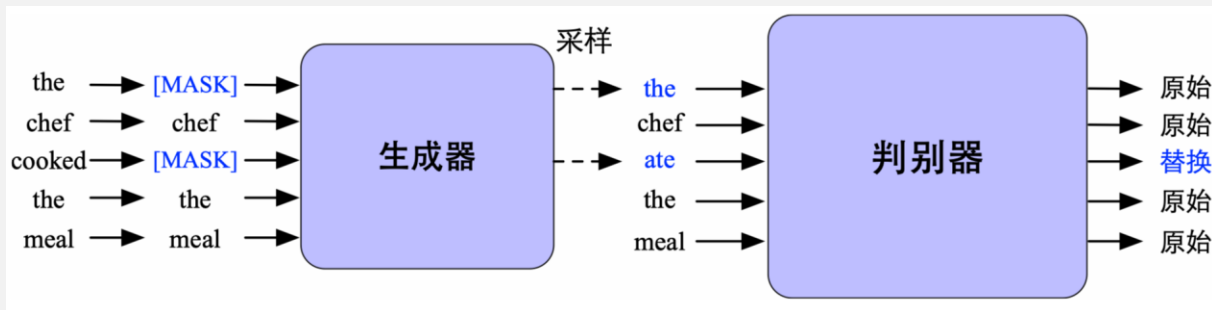
—





□ 概述

- 提出了一种全新基于生成器-判别器框架的预训练语言模型
- 相比传统预训练模型的训练效率更高
- 整体结构
 - 生成器-判别器框架，与GAN (Goodfellow et al., 2014) 类似
 - Generator: 将输入中的缺失信息还原为原单词
 - Discriminator: 判断输入的单词是否被替换过





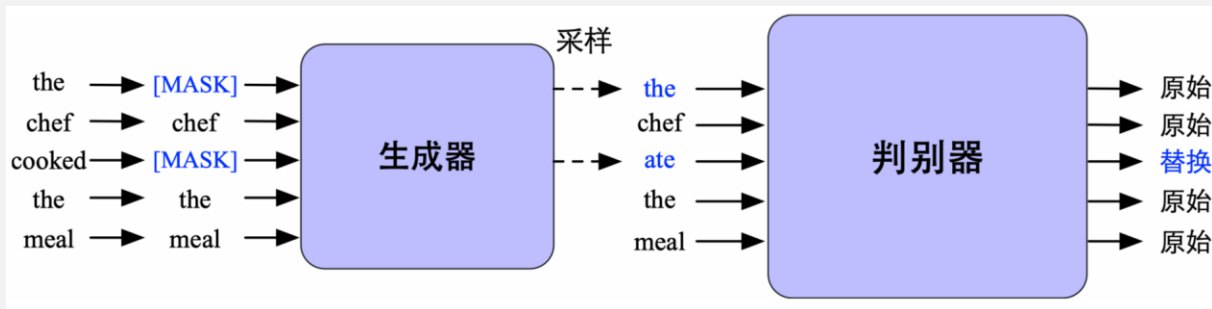
□ 模型结构

– 生成器：一个小的MLM

- 第一步：随机选取输入序列中的一部分进行遮蔽，通常比例选取15%
- 第二步：利用MLM任务，将缺失信息还原为原单词

– 判别器：常规的BERT结构

- 第一步：将缺失信息替换为生成器预测出来的单词
- 第二步：判别器学习判断输入句子中哪些单词被生成器替换过





□ 模型训练

- 生成器和判别器分别使用以下损失函数训练：

$$\mathcal{L}^G = - \sum_{i \in \mathbb{S}} \log P^G(x_i)$$
$$\mathcal{L}^D = - \sum_{i \in \mathbb{S}} [y_i \log P^D(x_i^s) + (1 - y_i) \log(1 - P^D(x_i^s))]$$

- 最终，模型通过最小化以下损失学习模型参数：

$$\min_{\theta^G, \theta^D} \sum_{x \in \mathcal{X}} \mathcal{L}^G(x, \theta^G) + \lambda \mathcal{L}^D(x, \theta^D)$$

- 注意：由于生成器和判别器衔接的部分涉及采样环节，判别器的损失并不会直接回传到生成器，因为采样操作是不可导的。另外，当预训练结束后，只需要使用判别器进行下游任务精调，而不再使用生成器。



□ 其他改进

- **更小的生成器**：生成器和判别器的主体结构均由BERT组成，因此两者完全可以使用同等大小的参数规模。但这样会导致预训练的时间大约为单个模型的两倍。为了提高预训练的效率，在ELECTRA中生成器的参数量要小于判别器。

类型	词向量维度	层数	隐含层维度	全连接层维度	注意力头数	注意力头维度
生成器	768	12	256	1,024	4	64
判别器	768	12	768	3,072	12	64

- **参数共享**：为了实现更灵活的建模目的，ELECTRA首先引入了词向量因式分解方法，通过全连接层将词向量维度映射到隐含层维度。由于ELECTRA使用了一个更小的生成器，因此生成器和判别器之间无法直接进行参数共享。在ELECTRA中，参数共享只限于输入层权重，其中包括词向量和位置向量矩阵。



□ 概述

- 在可比条件下，评测了主流预训练模型在中文下的表现
- 提出一种新的预训练模型MacBERT，解决“预训练-精调”不一致的问题
- 应用了一种基于文本纠错的掩码语言模型（MLM as correction, Mac）。

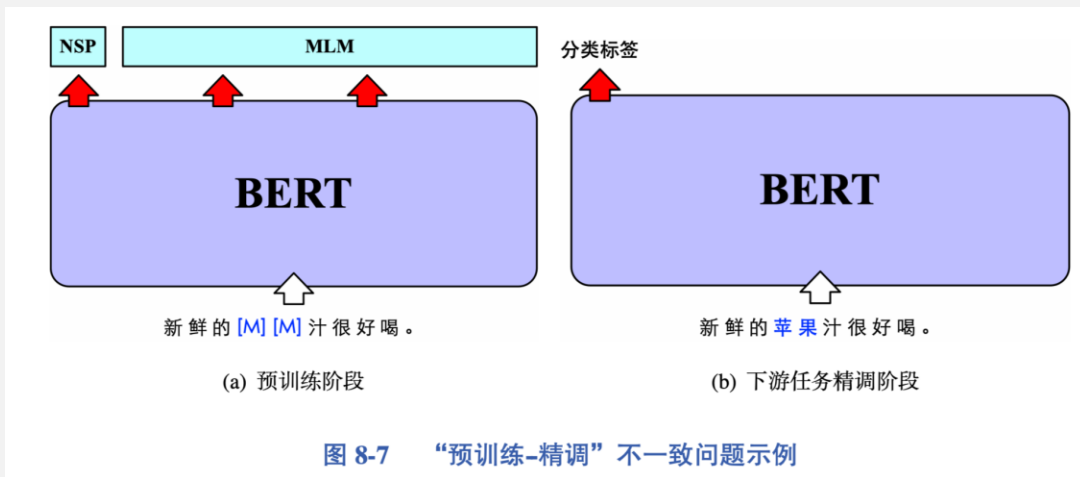


图 8-7 “预训练-精调”不一致问题示例



□ 预训练算法描述

- 使用了整词掩码和N-gram掩码，其中 unigram 至 4-gram 的概率分别为40%、30%、20% 和 10%；
- 为了解决掩码标记 [MASK] 在下游任务中不会出现的问题，在预训练阶段，MacBERT 使用相似词替换 [MASK] 标记。实际操作时，使用同义词词典获取待掩码单词的相似词。当 N-gram 掩码时，对 N-gram 中的每个词均进行相似词替换
- 与原版 BERT 类似，MacBERT 对输入序列总长度 15% 的标记进行掩码，在 80% 的情况下会替换为相似词，在 10% 的情况下会替换为随机词，在 10% 的情况下则不进行任何替换（负样本）。

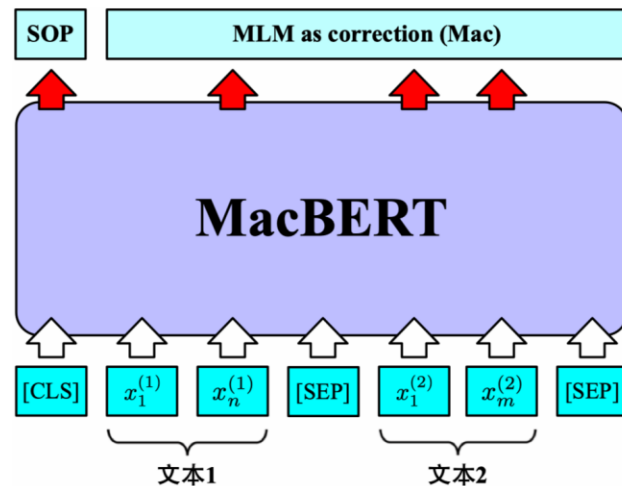


图 8-8 MacBERT 模型的整体结构

表 8-8 不同掩码方式的对比示例

原始句子	使用语言模型来预测下一个词的概率。
中文分词	使用语言模型来预测下一个词的概率。
原始掩码输入	使用语言[M]型来[M]测下一个词的概率。
整词掩码输入	使用语言[M][M]来[M][M]下一个词的概率。
N-gram 掩码输入	使用[M][M][M][M]来[M][M]下一个词的概率。
纠错型掩码输入	使用语法建模来预见下一个词的几率。



模型对比

模型	类型	切词	预训练任务	训练数据规模
BERT	自编码	WordPiece	MLM + NSP	≈ 16 GB
XLNet	自回归	SentencePiece	PLM	≈ 126 GB
RoBERTa	自编码	SentencePiece	MLM	≈ 160 GB
ALBERT	自编码	SentencePiece	MLM + SOP	≈ 16 GB
ELECTRA	自编码	WordPiece	Generator + Discriminator	≈ 126 GB
MacBERT	自编码	WordPiece	Mac + SOP	≈ 20 GB





大纲

- 模型优化
- 长文本处理
- 模型蒸馏与压缩
- 生成模型





□ 概述

- 以自注意力机制为核心的 Transformer 模型是各种预训练语言模型中的主要组成部分。自注意力机制能够构建序列中各个元素之间的上下文关联程度，挖掘深层次的语义信息。然而，**自注意力机制的时空复杂度为 $O(n^2)$** ，即时间和空间消耗会随着输入序列的长度呈平方级增长。这种问题的存在使得预训练语言模型处理长文本的效率较低。
- **传统处理长文本的方法一般是切分输入文本**，其中每份的大小设置为预训练语言模型能够单次处理的最大长度（如512）。最终将多片文本的决策结果进行综合（如对分类结果进行投票）或者拼接（如序列标注或生成任务）得到最终结果。然而，**这种方法不能很好地构建文本块之间的联系，挖掘长距离文本依赖的能力较弱**。因此，更好的方法还是需要**从根本上提高预训练语言模型单次能够处理的最大文本长度**，从而能够更加充分地利用自注意力机制。
- 接下来将介绍四个有代表性的擅长处理长文本序列的 Transformer 变种，即 Transformer-XL、Reformer、Longformer 和 BigBird。





Transformer-XL

□ 概述

- Transformer中处理长文本的传统策略是将文本切分成固定长度的块，并单独编码每个块，块与块之间没有信息交互
- 右图是一个块长度为4的一个示例
 - 训练阶段：Transformer分别对第一块中的序列 x_1 、 x_2 、 x_3 、 x_4 与第二块中的序列 x_5 、 x_6 、 x_7 、 x_8 进行建模
 - 测试阶段：由于每次处理的最大长度为4，当模型在处理序列 x_2 、 x_3 、 x_4 、 x_5 时，无法构建与历史 x_1 的关系
 - 另外，由于需要以滑动窗口的方式处理整个序列，这种方法的效率也非常低

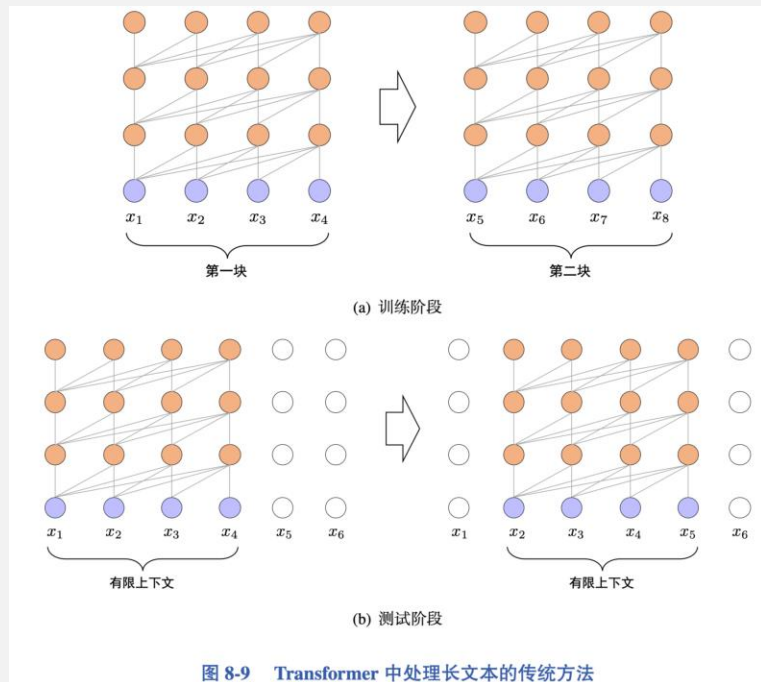


图 8-9 Transformer 中处理长文本的传统方法



Transformer-XL

□ 两种改进策略

– 右图是一个块长度为4的一个示例

- 状态复用的块级别循环

$$\begin{aligned}\tilde{h}_{\tau+1}^{[l-1]} &= [\text{SG}(h_{\tau}^{[l-1]}) \circ h_{\tau+1}^{[l-1]}] \\ q_{\tau+1}^{[l]} &= h_{\tau+1}^{[l-1]} W^q{}^\top \quad k_{\tau+1}^{[l]} = \tilde{h}_{\tau+1}^{[l-1]} W^k{}^\top \quad v_{\tau+1}^{[l]} = \tilde{h}_{\tau+1}^{[l-1]} W^v{}^\top \\ h_{\tau+1}^{[l]} &= \text{Transformer-Block}(q_{\tau+1}^{[l]}, k_{\tau+1}^{[l]}, v_{\tau+1}^{[l]})\end{aligned}$$

- 相对位置编码

$$a_{i,j} = \underbrace{v_{x_i}^\top W^q{}^\top W^E v_{x_j}}_{(a)} + \underbrace{v_{x_i}^\top W^q{}^\top W^R R_{i-j}}_{(b)} + \underbrace{u_{x_i}^\top W^E v_{x_j}}_{(c)} + \underbrace{u_{x_i}^\top W^R R_{i-j}}_{(d)}$$

- 基于内容的相关度 (a)
- 内容相关的位置偏置 (b)
- 全局内容偏置 (c)
- 全局位置偏置 (d)

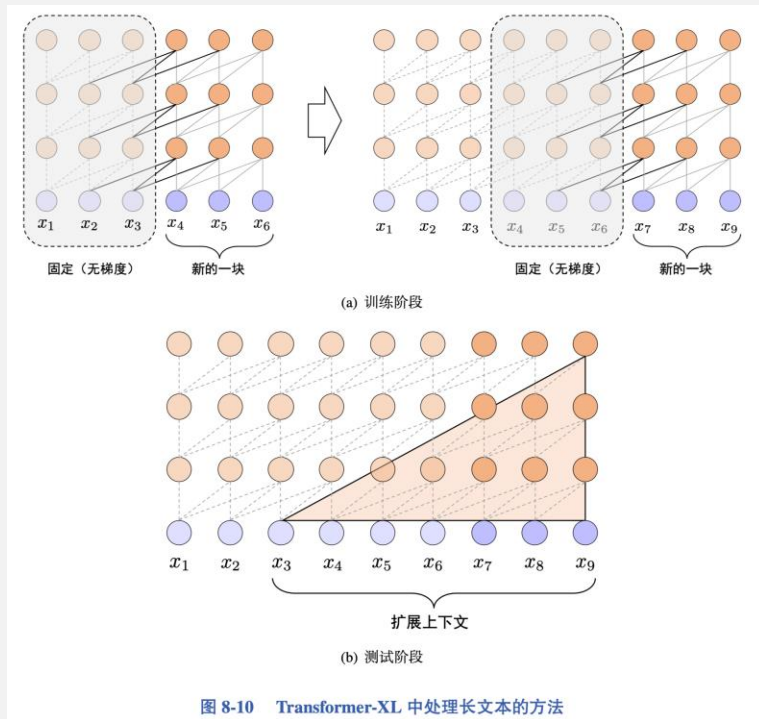


图 8-10 Transformer-XL 中处理长文本的方法



□ 概述

- 引入了局部敏感哈希注意力和可逆 Transformer 技术，有助于减少模型的内存占用，进一步提升了模型对长文本的处理能力
- 问题①：单独计算查询和键的必要性
 - 作者通过实验证实，查询和键相同的Transformer 与传统的 Transformer 相比并没有太大的性能差异
 - 在Reformer中采用了 QK共享的 Transformer，减少了注意力机制中的一部分计算
- 问题②：全局注意力计算的必要性
 - 通过 Softmax 函数得到的结果主要取决于数值较大的若干元素，因此并不需要将所有的词都参与到注意力的计算中
 - 如果只计算那些与当前查询关联度最高的 n 个词，就可以极大地降低注意力计算量



□ 局部敏感哈希

- 通过局部敏感哈希算法（LSH）高效地计算与每个词关联度最高的 n 个词

为了得到 b 个哈希值，定义一个随机矩阵 $R \in \mathbb{R}^{d \times b/2}$ （ d 表示隐含层维度），并定义哈希函数：

$$h(x) = \arg \max([xR; -xR])$$

只需要对桶内所有的元素进行注意力的计算就能够达到近似完整注意力机制的目的，极大地降低了注意力计算的复杂度。

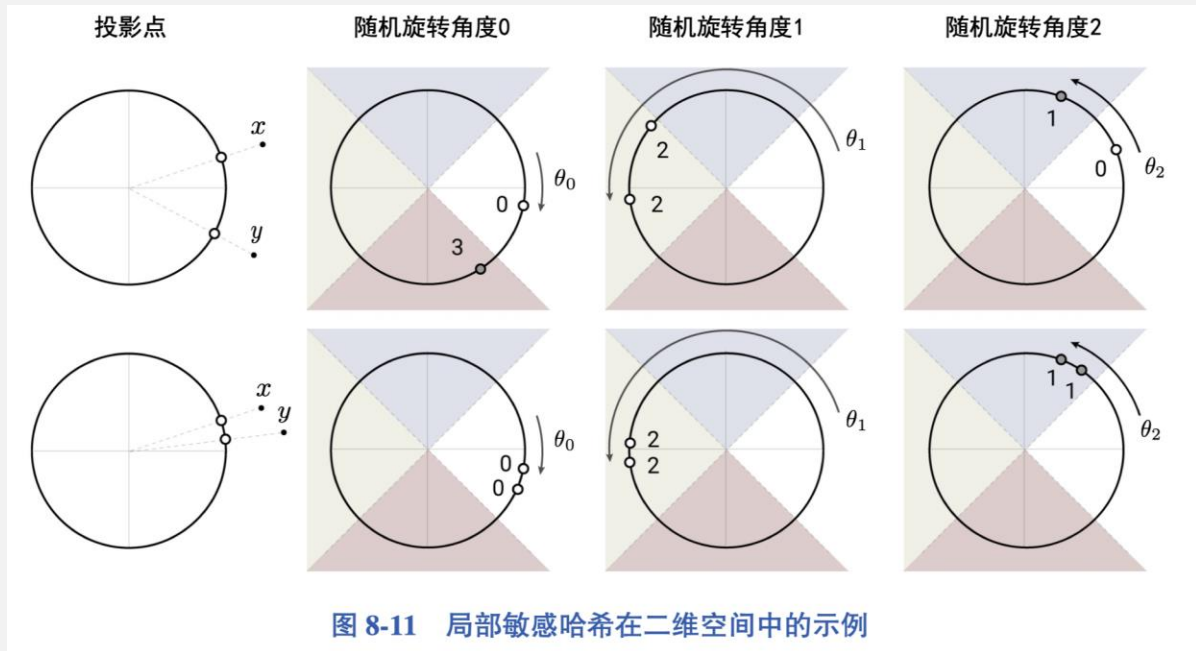


图 8-11 局部敏感哈希在二维空间中的示例



□ 局部敏感哈希注意力的计算

- 在实际应用中，通常使用并行计算加快模型的计算速度。
- 不过，每个桶内包含的向量数目可能不尽相同，这里并不能只计算单个桶内的注意力，还需要多加入一个桶参与计算。
- 最后进入注意力计算的环节，每个元素只会与当前和前一个块中具有相同哈希值的元素计算注意力。
- 局部敏感哈希会以小概率出现失败，使用**多轮局部敏感哈希**可进一步降低错误率

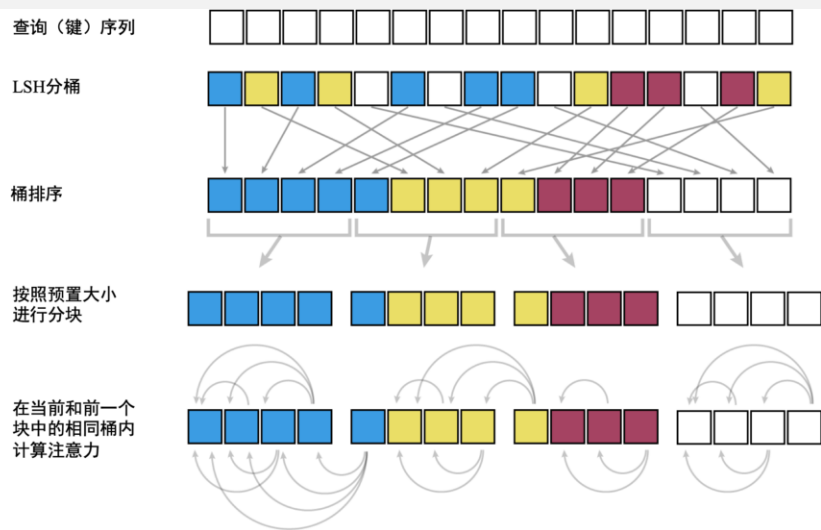


图 8-12 局部敏感哈希注意力的计算示例



□ 局部敏感哈希注意力的计算

- 在实际应用中，通常使用并行计算加快模型的计算速度。
- 不过，每个桶内包含的向量数目可能不尽相同，这里并不能只计算单个桶内的注意力，还需要多加入一个桶参与计算。
- 最后进入注意力计算的环节，每个元素只会与当前和前一个块中具有相同哈希值的元素计算注意力。
- 局部敏感哈希会以小概率出现失败，使用**多轮局部敏感哈希**可进一步降低错误率

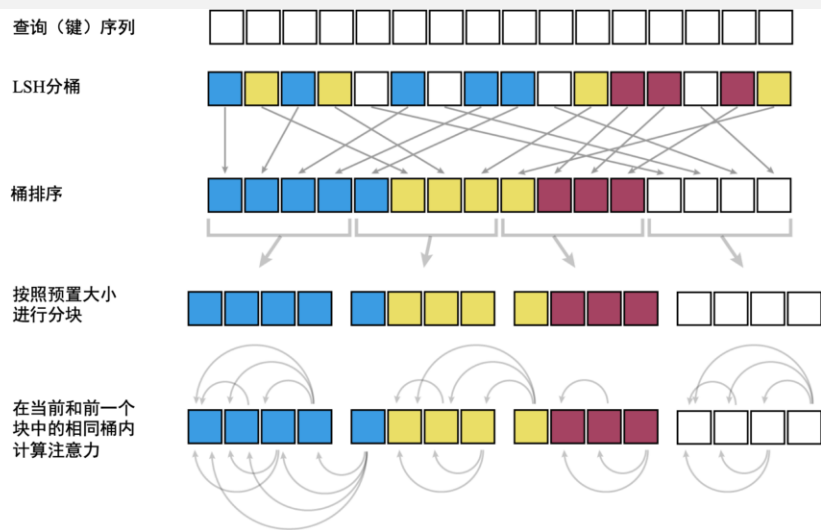


图 8-12 局部敏感哈希注意力的计算示例

2轮能达到 76.9%，而8轮能够达到 94.8%

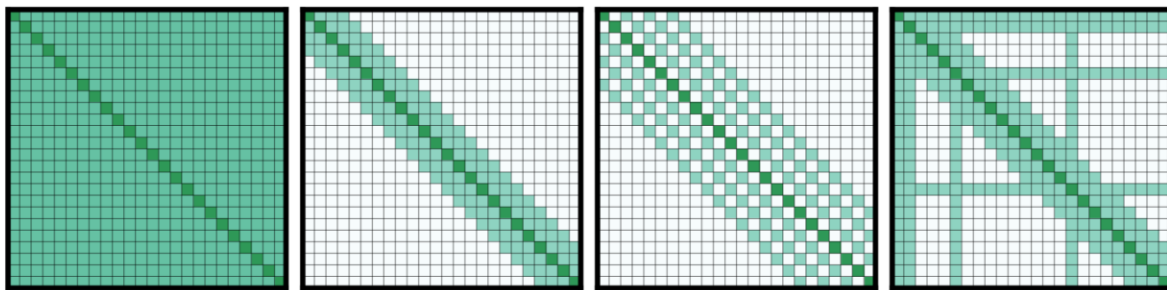




Longformer

□ 概述

- 模型基于稀疏注意力机制，最大可处理长度扩展至4096
- 三种稀疏注意力模式
 - 滑动窗口注意力：每个词只会与其相邻的 k 个词计算注意力
 - 扩张滑动窗口注意力：每间隔 $d-1$ 采样一次
 - 全局+滑动窗口注意力：使一些预先选定的位置能够看到全局信息



(a) 完整的注意力

(b) 滑动窗口注意力

(c) 扩张滑动窗口注意力

(d) 全局 + 滑动窗口

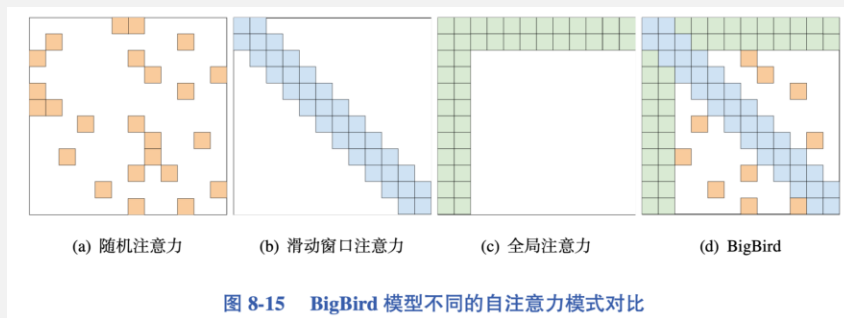
文本分类任务：
将[CLS] 设置为“全局可见”

问答类任务：
将所有的问题中的单词设置
为“全局可见”



□ 概述

- 模型同样借鉴了稀疏注意力的方法
- BigBird结合了以下三种不同注意力模式
 - 随机注意力: 针对每一个词, 随机选取 r 个词参与注意力的计算;
 - 滑动窗口注意力: 与 Longformer 相同, 即只利用当前词周围的 k 个词计算注意力;
 - 全局注意力: 与 Longformer 基本相同
 - 内部 Transformer 组建模式: 从输入序列中选择 g 个词, 使其能够见到所有词, 反之亦然。
 - 外部 Transformer 组建模式: 在输入序列中插入额外的全局标记, 使其能够见到所有词, 反之亦然。





模型对比

模型	复杂度	主要思想
Set Transformer ^[47]	$\mathcal{O}(nk)$	记忆机制
Transformer-XL ^[28]	$\mathcal{O}(n^2)$	循环机制
Sparse Transformer ^[48]	$\mathcal{O}(n\sqrt{n})$	固定模式
Reformer ^[40]	$\mathcal{O}(n \log n)$	可学习模式
Longformer ^[44]	$\mathcal{O}(n(k+m))$	固定模式
ETC ^[49]	$\mathcal{O}(n_g^2 + nn_g)$	固定模式
Synthesizer ^[50]	$\mathcal{O}(n^2)$	可学习模式
Performer ^[51]	$\mathcal{O}(n)$	核 (Kernel) 方法
Linformer ^[52]	$\mathcal{O}(n)$	低秩 (Low rank) 方法
Linear Transformers ^[53]	$\mathcal{O}(n)$	核方法
BigBird ^[45]	$\mathcal{O}(n)$	固定模式





大纲

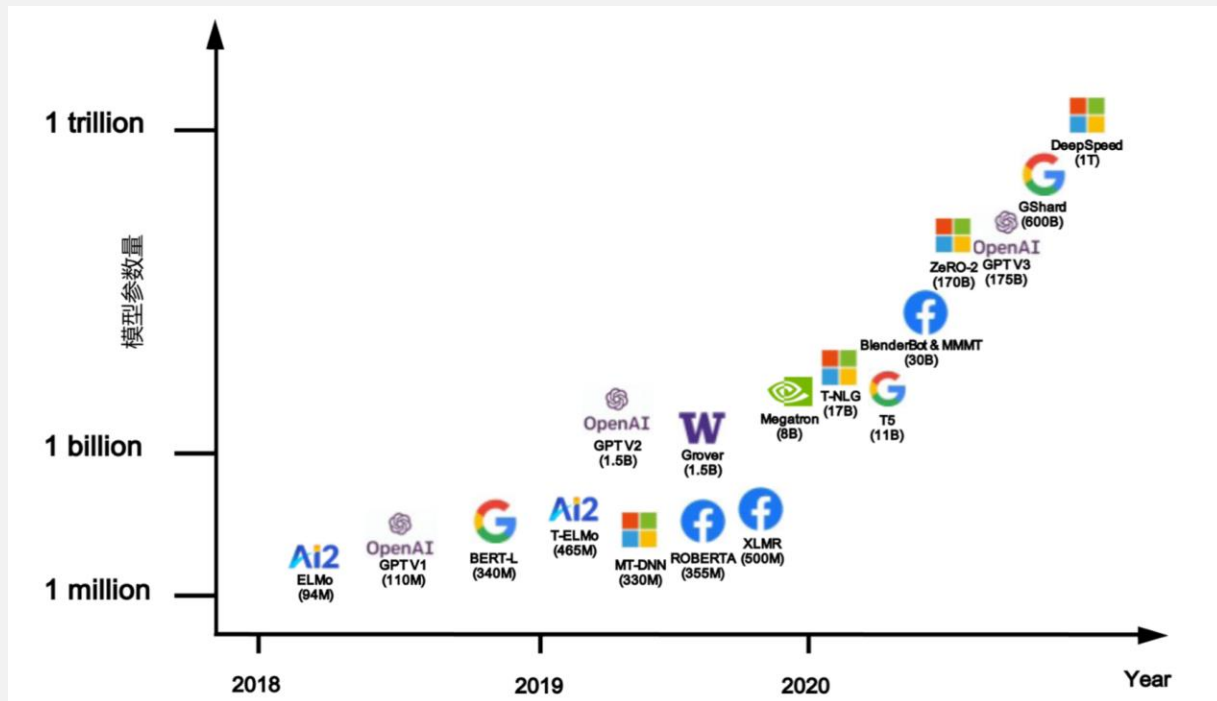
- 模型优化
- 长文本处理
- 模型蒸馏与压缩
- 生成模型





模型蒸馏与压缩

□ 预训练模型参数量趋势

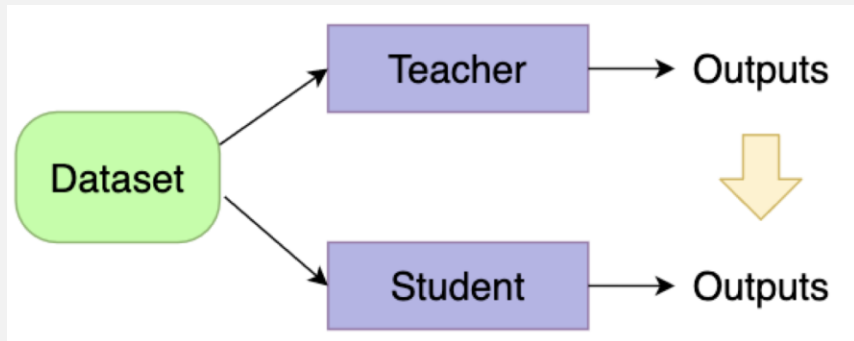




模型蒸馏与压缩

□ 为什么需要模型压缩与知识蒸馏技术？

- 预训练模型通常需要占用很大的空间，并且训练和推断时间也很慢
- 直接在实际产品或应用中使用预训练模型难以满足时间和空间需求
- **知识蒸馏技术**可以在不损失或少量损失性能的情况下，将大模型的知识迁移到小模型，从而提升推断速度



知识蒸馏过程：“老师教学生”

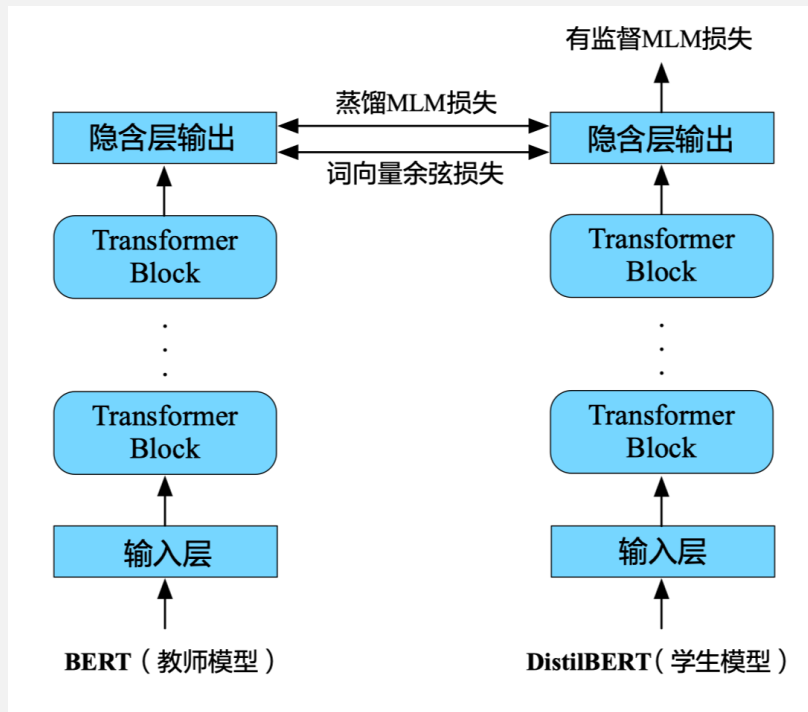




模型蒸馏与压缩

□ DistilBERT

- 通用、任务无关的6层BERT
- 相比BERT-base, 小40%、快60%、在NLU任务上可达到原模型的97%
- 使用MLM预训练任务进行知识蒸馏(无NSP)
- 训练目标由以下三部分组成
 - 由数据集自带的硬标签(hard-labels)计算的有监督MLM损失
 - 由教师模型提供的软标签(soft-labels)计算的蒸馏MLM损失
 - 教师模型和学生模型隐层输出之间的余弦相似度损失





□ DistilBERT

- 总损失函数

$$\mathcal{L} = \mathcal{L}^{\text{s-mlm}} + \mathcal{L}^{\text{d-mlm}} + \mathcal{L}^{\text{cos}}$$

- 有监督MLM损失

$$\mathcal{L}^{\text{s-mlm}} = - \sum_i y_i \log(s_i)$$

- 蒸馏MLM损失

$$\mathcal{L}^{\text{d-mlm}} = - \sum_i t_i \log(s_i)$$

- 词向量余弦损失

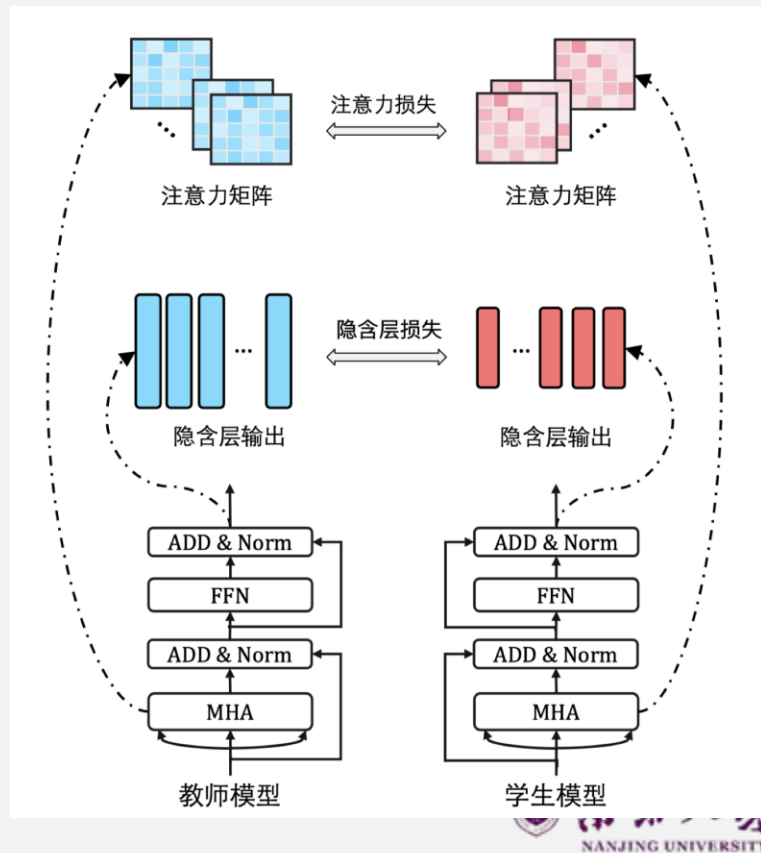
$$\mathcal{L}^{\text{cos}} = \cos(\mathbf{h}^t, \mathbf{h}^s)$$



模型蒸馏与压缩

□ TinyBERT

- 提出了一种对BERT不同层进行匹配的蒸馏策略
- 提出了两阶段蒸馏策略，在预训练和精调阶段均进行知识蒸馏
- TinyBERT能达到教师模型BERT-base的96%的效果（GLUE），大小只有教师的13.3%
- 蒸馏损失由3部分组成
 - 词向量损失
 - 中间层损失
 - 预测层损失





模型蒸馏与压缩

□ TinyBERT

- 总损失函数
- 词向量层损失
- 中间层损失
- 预测层损失

$$\mathcal{L} = \mathcal{L}^{\text{emb}} + \mathcal{L}^{\text{mid}} + \mathcal{L}^{\text{pred}}$$

$$\mathcal{L}^{\text{emb}} = \text{MSE}(\mathbf{v}^s \mathbf{W}^e, \mathbf{v}^t)$$

$$\mathcal{L}^{\text{hid}}(i, j) = \text{MSE}(\mathbf{h}^{s[i]} \mathbf{W}^h, \mathbf{h}^{t[j]}), \quad \text{s.t. } g(i) = j$$

$$\mathcal{L}^{\text{att}}(i, j) = \frac{1}{K} \sum_{k=1}^K \text{MSE}(\mathbf{A}^{s[i]}, \mathbf{A}^{t[j]}), \quad \text{s.t. } g(i) = j$$

$$\mathcal{L}^{\text{mid}} = \sum_{i,j} [\mathcal{L}^{\text{hid}}(i, j) + \mathcal{L}^{\text{att}}(i, j)], \quad \text{s.t. } g(i) = j$$

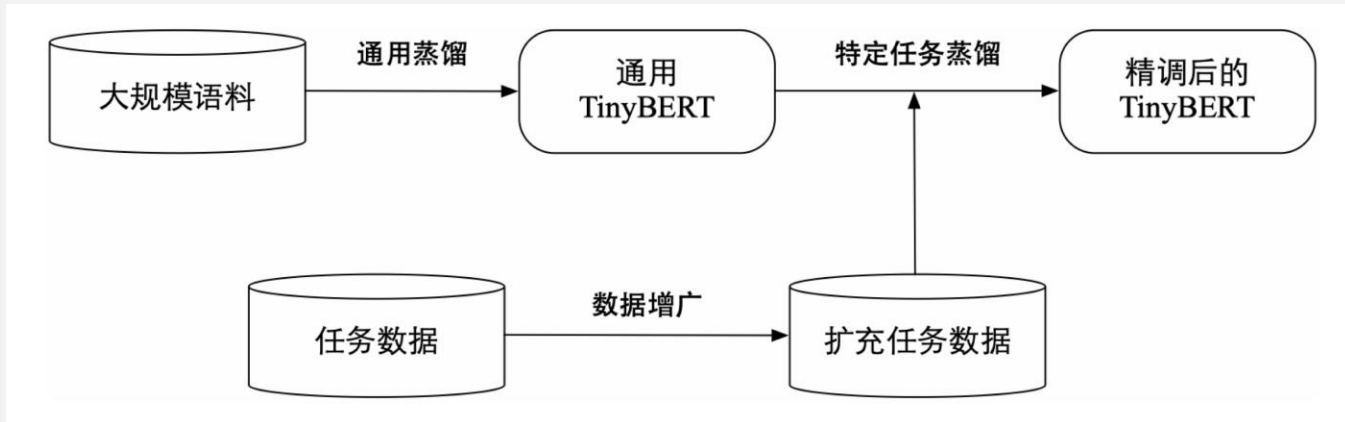
$$\mathcal{L}^{\text{pred}} = \sum_i t_i \log(s_i)$$



模型蒸馏与压缩

□ TinyBERT: 两段式蒸馏

- 通用蒸馏：利用原始BERT作为教师，并使用大规模文本训练MLM任务
- 特定任务蒸馏：使用精调过的BERT作为教师，使用数据增广后的任务数据进行蒸馏

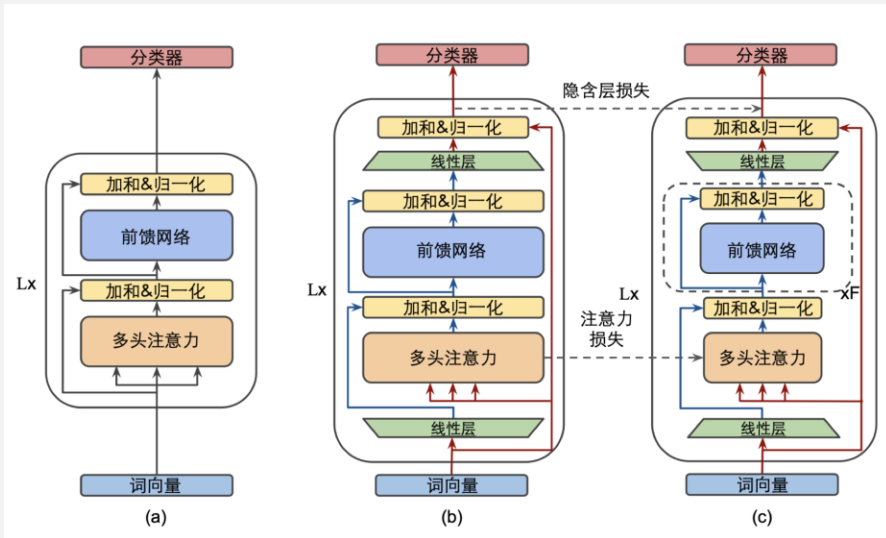




模型蒸馏与压缩

□ MobileBERT

- MobileBERT与BERT-large深度相同但更“苗条”
- 在自注意力和前馈神经网络的设计上也有一定的改进
- 能够达到教师模型（BERT-base）99.2%的性能效果（以GLUE为测试基准），推理速度快5.5倍，参数量降低至23.2%
- 蒸馏损失由3部分组成
 - MLM损失
 - 隐含层匹配损失
 - 注意力匹配损失（KL散度）

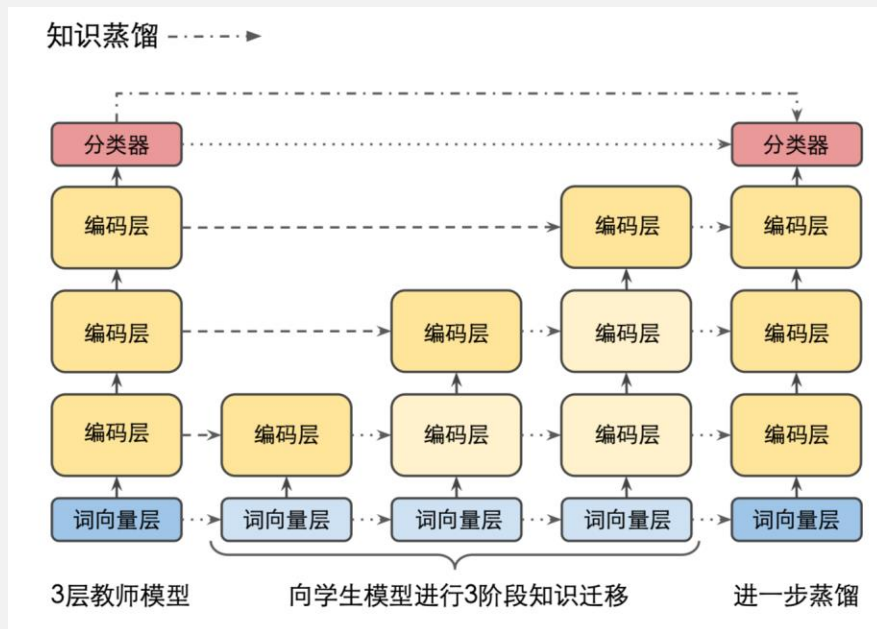




模型蒸馏与压缩

□ MobileBERT

- 渐进式知识迁移（Progressive Knowledge Transfer）
 - 词向量层和最终分类输出层的权重是直接从未教师模型拷贝至学生模型的，始终不参与参数更新
 - 对于中间的 Transformer 层，采用了渐进的方式逐步训练
 - 当学生模型学习教师模型的第 i 层时，学生模型中所有小于 i 层的权重均不参与更新





模型蒸馏与压缩

□ TextBrewer: An Open-Source Knowledge Distillation Toolkit

- 推出了首个面向自然语言处理领域的基于Pytorch的知识蒸馏工具包
- 提供了方便、快捷、易用的知识蒸馏框架，少量性能损失换取大幅速度提升
- <http://textbrewer.hfl-rc.com/> 或通过 `pip install textbrewer` 安装



TextBrewer

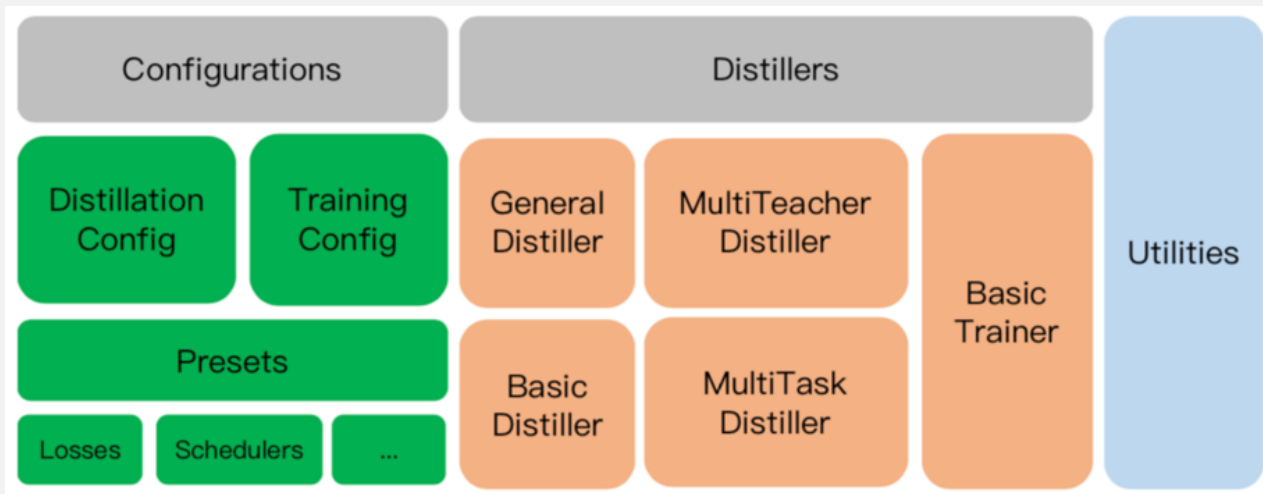
- 模型无关：适用于多种模型结构（主要面向Transformer结构）
- 方便灵活：可自由组合多种蒸馏方法，支持增加自定义损失等模块
- 非侵入式：无需对教师与学生模型本身结构进行修改
- 适用面广：支持典型NLP任务，如文本分类、阅读理解、序列标注等



模型蒸馏与压缩

□ TextBrewer: 整体架构

- Distillers: 用于执行实际的知识蒸馏工作
- Configurations: 为Distillers提供必要的配置
- Utilities: 包含一些辅助的功能，如模型参数统计等



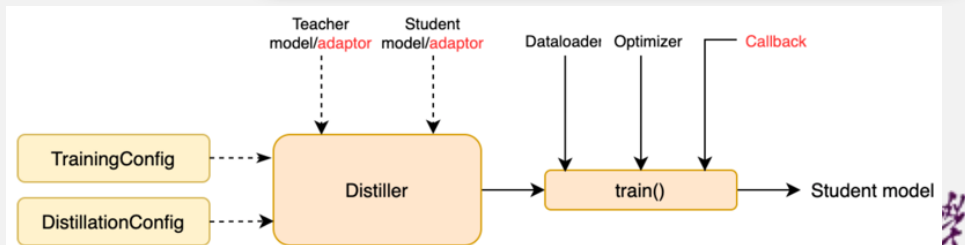


模型蒸馏与压缩

□ TextBrewer: 工作流程

- 第一步：开始蒸馏之前的准备工作
 - 训练教师模型，定义并初始化学生模型
 - 构造蒸馏用数据集的DataLoader
- 第二步：知识蒸馏
 - 初始化Distiller，构造训练配置和蒸馏配置
 - 定义adaptors和callback，分别用于适配模型输入输出和训练过程中的回调
 - 调用Distiller的train方法开始蒸馏
- 一共只需20行左右的代码！

```
1 from textbrewer import GeneralDistiller
2 from textbrewer import TrainingConfig, DistillationConfig
3
4 # We omit the initialization of models, optimizer, and dataloader.
5 teacher_model : torch.nn.Module = ...
6 student_model : torch.nn.Module = ...
7 dataloader : torch.utils.data.DataLoader = ...
8 optimizer : torch.optim.Optimizer = ...
9 scheduler : torch.optim.lr_scheduler = ...
10
11 def simple_adaptor(batch, model_outputs):
12     # We assume that the first element of model_outputs
13     # is the logits before softmax
14     return {'logits': model_outputs[0]}
15
16 train_config = TrainingConfig()
17 distill_config = DistillationConfig()
18 distiller = GeneralDistiller(
19     train_config=train_config, distill_config=distill_config,
20     model_T = teacher_model, model_S = student_model,
21     adaptor_T = simple_adaptor, adaptor_S = simple_adaptor)
22
23 distiller.train(optimizer, scheduler,
24                 dataloader, num_epochs, callback=None)
```





模型蒸馏与压缩

□ TextBrewer: 实验效果

- 教师模型: BERT-base (110M)
- 学生模型: T6 (60%), T3 (41%), T3-small (16%), T4-tiny (与TinyBERT相同大小, 13%)
- 单教师知识蒸馏
 - T6结构可以达到教师模型效果的99%, 且模型体积缩小至60%
 - T4-tiny知识蒸馏结果优于TinyBERT
- 多教师知识蒸馏
 - 所有模型使用同样的结构, 即BERT-base
 - 蒸馏后的学生模型获得最优效果, 且超过简单的模型融合 (ensemble)

Model	MNLI		SQuAD		CoNLL-2003
	m	mm	EM	F1	F1
BERT _{BASE}	83.7	84.0	81.5	88.6	91.1
<i>Public</i>					
DistilBERT	81.6	81.1	79.1	86.9	-
TinyBERT	80.5	81.0	-	-	-
+DA	82.8	82.9	72.7	82.1	-
<i>TextBrewer</i>					
BiGRU	-	-	-	-	85.3
T6	83.6	84.0	80.8	88.1	90.7
T3	81.6	82.5	76.3	84.8	87.5
T3-small	81.3	81.7	72.3	81.4	78.6
T4-tiny	82.0	82.6	73.7	82.5	77.5

Model	MNLI		SQuAD		CoNLL-2003
	m	mm	EM	F1	F1
Teacher 1	83.6	84.0	81.1	88.6	91.2
Teacher 2	83.6	84.2	81.2	88.5	90.8
Teacher 3	83.7	83.8	81.2	88.7	91.3
Ensemble	84.3	84.7	82.3	89.4	91.5
Student	84.8	85.3	83.5	90.0	91.6



大纲

- 模型优化
- 长文本处理
- 模型蒸馏与压缩
- 生成模型

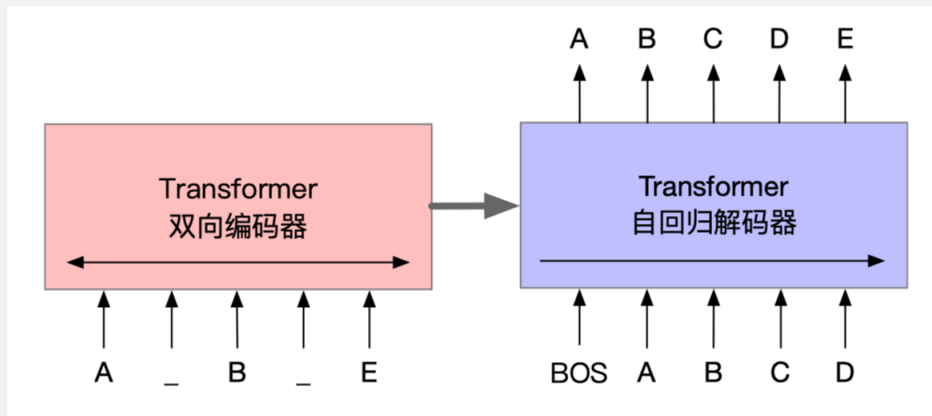




生成模型

□ BART

- 采用了经过细微调整的基于Transformer的序列到序列结构
- 结合双向的Transformer 编码器与单向的自回归 Transformer 解码器，通过对含有噪声的输入文本去噪重构进行预训练，是一种典型的去噪自编码器（Denoising autoencoder）
- 训练过程：双向编码器编码带噪音的文本，用单向自回归编码器重构文本

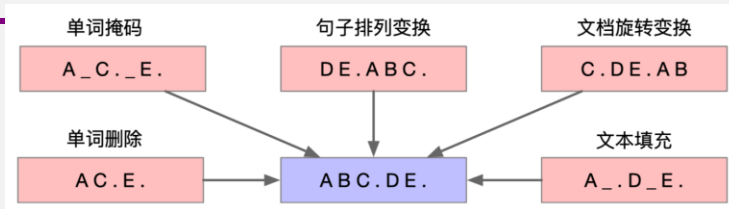




生成模型

□ BART: 预训练任务

- 单词掩码
 - 与BERT类似，在输入文本中随机采样一部分单词，并替换为掩码标记（如[MASK]）；
- 单词删除
 - 随机采样一部分单词并删除。要处理这类噪声，模型不仅需要预测缺失的单词，还需要确定缺失单词的位置；
- 句子排列变换
 - 将打乱顺序的句子还原，模型需要对整段输入文本的语义具备一定理解；
- 文档旋转变换
 - 随机选择输入文本中的一个单词，并旋转文档，使其以该单词作为开始。为了重构原始文本，模型需要从扰乱文本中找到原始文本的开头；
- 文本填充
 - 随机采样多个文本片段，片段长度根据泊松分布（ $\lambda=3$ ）进行采样得到。用单个掩码标记替换每个文本片段。当片段长度为0时，意味着插入一个掩码标记。要去除这类噪声，要求模型具有预测缺失文本片段长度的能力。

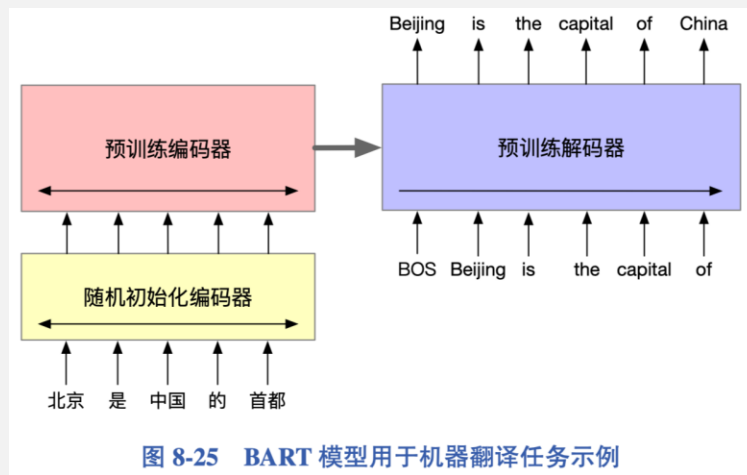




生成模型

□ BART: 模型精调

- 序列分类
 - 编码器解码器使用相同输入，使用解码器最终时刻隐含层状态进行分类
- 序列标注
 - 编码器解码器使用相同输入，使用解码器每个时刻的隐含层状态进行分类（标注）
- 文本生成
 - 编码器的输入是作为条件的输入文本，解码器则以自回归的方式生成对应的目标文本
- 机器翻译
 - 将BART模型编码器的输入表示层替换为一个小型Transformer编码器，用来将源语言中的词汇映射至目标语言的输入表示空间，从而适配BART模型的预训练环境

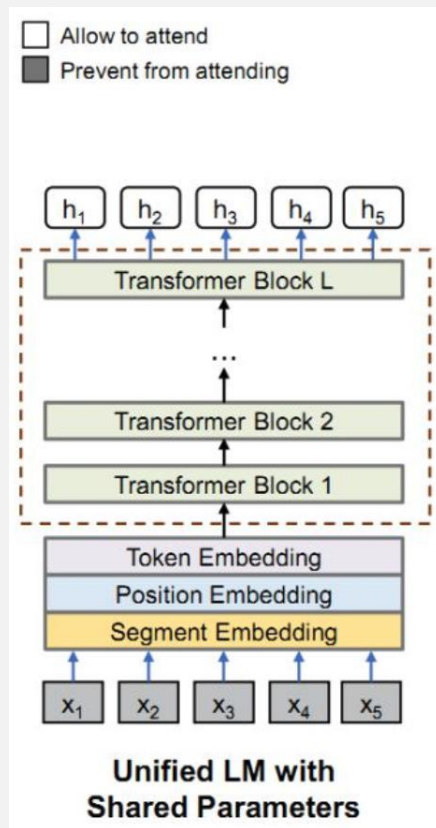




生成模型

□ UniLM

- 只用一个Transformer就可以同时完成语言表示和文本生成的预训练
- 可同时应用于语言理解任务和文本生成任务
- 核心思想：使用不同的自注意力掩码矩阵，控制每个词的注意力范围，从而实现不同语言模型对于信息流的控制

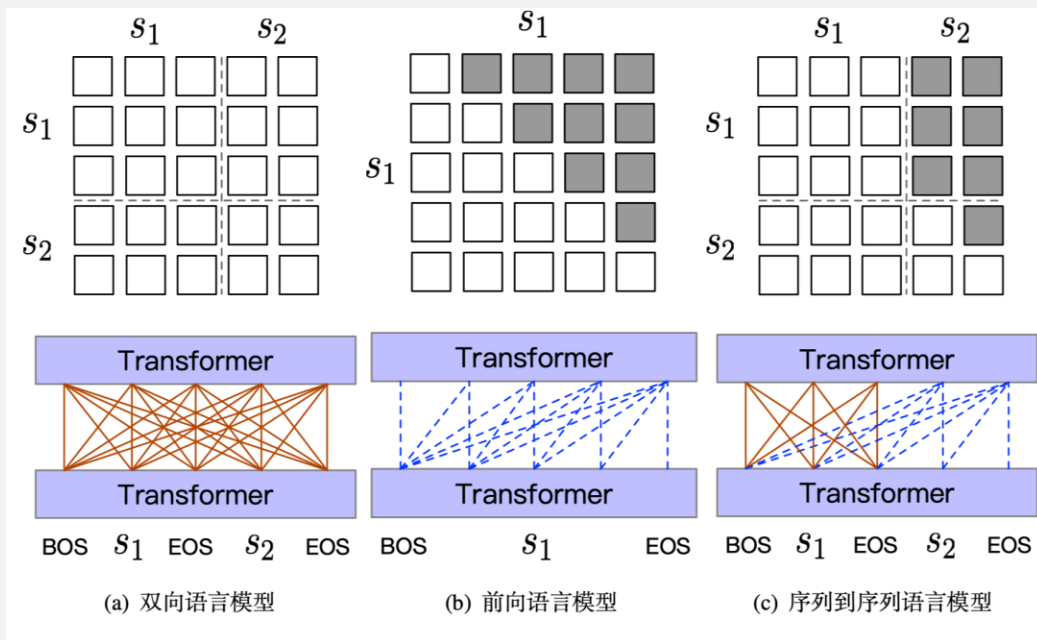




生成模型

□ UniLM

- 可以利用双向语言模型、单向语言模型和序列到序列语言模型进行预训练

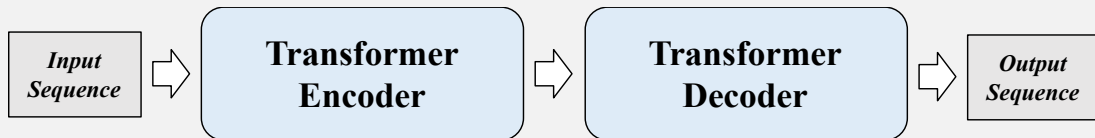
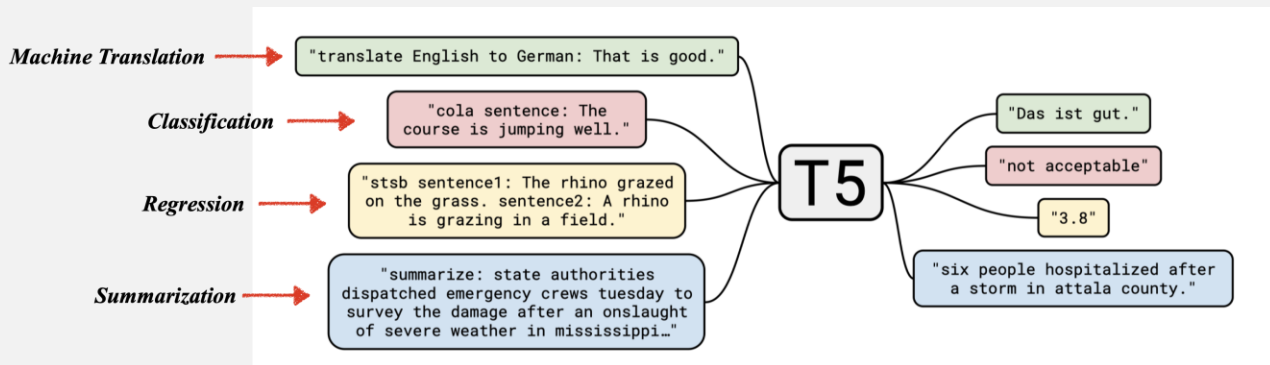




生成模型

□ T5: Text-to-Text Transfer Transformer

- 提出一种适用于各种类型NLP任务的Encoder-Decoder框架
- 提供了非常细致的模型设计决策过程
- 提出了C4数据集，包含750G高质量英文数据





生成模型

□ T5: Text-to-Text Transfer Transformer

- 提出了一种基于span-corruption的无监督训练任务
- 输入：对句子中的若干文本片段进行mask
- 输出：预测被mask的文本片段

Original text

Thank you ~~for inviting~~ me to your party ~~last~~ week.

Inputs

Thank you <X> me to your party <Y> week.

Targets

<X> for inviting <Y> last <Z>



□ GPT-3: Language Models are Few-Shot Learners

- 展示了超大规模语言模型在小样本学习（few-shot learning）上的能力
- 模型参数量进一步扩展至175B，预示着预训练模型进入到超大规模时代

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}



□ GPT-3: Language Models are Few-Shot Learners

- 传统预训练模型：直接在下游任务数据上精调
- GPT-3类超大规模模型: zero-shot, one-shot, few-shot
- 例如，对于机器翻译任务，在小样本的情况下，为了获得“cheese”的法语翻译，可以构建以下输入

```
Translate English to French:  
sea otter => loutre de mer  
plush girafe => girafe peluche  
cheese =>
```





□ 可控文本生成：CTRL

- 可以根据指定的领域、风格、主题、实体和实体关系等属性生成相应文本
- 从结构上仍然是一个基于Transformer 的自回归语言模型
- 核心思想是从海量无标注数据中定位文章所在的领域或其他属性，并作为控制代码（Control codes）放在输入文本的头部，以指导后续文本的生成
- 例如，需要生成一段关于“刀”（Knife）且具有有“恐怖”（Horror）色彩的文本，只需要以“Horror A knife”作为提示（Prompt）

Horror A knife handle pulled through the open hole in the front. I jumped when the knife hit.\n\nEyes widened in horror. Her scream was the only sound I heard besides her sobs.\n\nThe spider touched her feet as it started to dig into the top of her arch. The creature's tears began to flow.\n\nThe spider looked up to her and looked back at me with eyes filled with burning tears. My heart started to race ...



□ 可控文本生成：CTRL

- 提供了一种无须重新训练，且即插即用的方法实现可控的文本生成
- 核心思想是，对于预训练语言模型（GPT-2）以及目标属性 a （例如情感、主题等），利用当前的生成结果是否满足属性 a （即条件概率 $P(a | x)$ ）对生成进行修正，使其朝着满足该属性的方向变化
- 具体修正过程可分为以下三个步骤：
 - 前向过程：包括语言模型以及属性判别（即 $P(a | x)$ 的计算）；
 - 反向过程：利用属性判别模型回传的梯度，更新语言模型内部的历史状态，使得实际预测更接近目标属性；
 - 重采样：根据新的概率分布，重新采样下一个生成的词。



小结

- 模型优化
- 长文本处理
- 模型蒸馏与压缩
- 生成模型

